SOFTWARE DESIGN SPECIFICATION

for

Hostel Management System (HMS)

Version 1.0

Prepared by: Team 42
Instructors: Sai Srithaja & Arun Avinash Chauhan
Date: April 2026

Document Information

Title:	Hostel Management System – Software Design Specification	
Project Manager:	Team 42	Version: 1.0 | Date: April 2026
Prepared By:	Team 42 Members	Preparation Date: April 2026

Version History

Ver. No.	Ver. Date	Revised By	Description	Filename
1.0	April 2026	Team 42	Initial Release	HMS_SDS_v1.0.docx

1. Introduction

This document provides a comprehensive Software Design Specification (SDS) for the Hostel Management System (HMS). It describes the system architecture, detailed design of modules, data model, exception handling strategy, and quality of service considerations.

1.1 Purpose

The purpose of this SDS is to describe the internal design of the Hostel Management System, which automates hostel operations including room allocation, complaint management, ticket management, student registration, notifications, and the notice board. This document is intended for the development team (Team 42), academic evaluators, and instructors Sai Srithaja and Arun Avinash Chauhan.

1.2 Scope

This SDS applies to the Hostel Management System (HMS), a web-based application built using React (frontend), Node.js + Express (backend), and MySQL (database). The system covers the following modules:

- Room Allocation Management
- Complaint Management
- Ticket Management
- Student Registration
- Notification System
- Notice Board

The following are explicitly OUT OF SCOPE: fee payment processing and room booking by students.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
HMS	Hostel Management System
SDS	Software Design Specification
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token – used for authentication
CRUD	Create, Read, Update, Delete operations
UI	User Interface
DB	Database (MySQL in this project)
React	JavaScript library for building user interfaces

Term / Acronym	Definition
Node.js	JavaScript runtime for the backend server
Express	Node.js web application framework
MySQL	Relational Database Management System used for persistence

1.4 References

- HMS Software Requirements Specification (HMS_SRS_Complete.docx) – Team 42, 2026
- React Documentation – https://react.dev
- Node.js Documentation – https://nodejs.org
- Express.js Documentation – https://expressjs.com
- MySQL Documentation – https://dev.mysql.com/doc/
- IEEE Std 1016-2009: IEEE Standard for Information Technology – Systems Design – Software Design Descriptions

2. Use Case View

This section identifies and describes the key use cases of the HMS. These use cases represent the primary interactions between users and the system and form the basis for the design decisions described in subsequent sections.

2.1 Use Cases

UC-01: Student Registration

Field	Detail
Actor	Admin
Description	Admin registers a new student into the hostel system
Precondition	Admin is authenticated and logged in
Steps	1. Admin opens Student Registration form 2. Enters student details (name, roll no, contact, room preference) 3. Submits form 4. System validates and stores record in DB 5. Confirmation displayed
Postcondition	Student record created; room not yet allocated

UC-02: Room Allocation

Field	Detail
Actor	Admin
Description	Admin allocates a hostel room to a registered student
Precondition	Student is registered; room is available
Steps	1. Admin views available rooms 2. Selects student and room 3. Confirms allocation 4. System updates room status and creates allocation record
Postcondition	Room marked as occupied; student linked to room

UC-03: Complaint Submission

Field	Detail
Actor	Student
Description	Student submits a complaint regarding hostel facilities
Precondition	Student is registered and authenticated
Steps	1. Student navigates to Complaints section 2. Fills in complaint title, description, category 3. Submits complaint 4. System records complaint with PENDING status
Postcondition	Complaint stored; admin notified

UC-04: Ticket Management

Field	Detail
Actor	Admin / Staff
Description	Admin reviews and resolves maintenance tickets raised from complaints or directly
Precondition	Ticket exists in the system
Steps	1. Admin views open tickets 2. Assigns ticket to staff 3. Staff updates ticket status (IN_PROGRESS, RESOLVED) 4. System logs status change
Postcondition	Ticket closed; student notified

UC-05: Notice Board Management

Field	Detail
Actor	Admin
Description	Admin posts notices/announcements to all hostel residents
Precondition	Admin is authenticated
Steps	1. Admin opens Notice Board module 2. Enters notice title and body 3. Publishes notice 4. System stores and displays notice to all students

Field	Detail
Postcondition	Notice visible to all registered students

UC-06: Notification System

Field	Detail
Actor	System (automated)
Description	System sends in-app notifications to students on relevant events
Precondition	Triggering event occurs (e.g., complaint resolved, notice posted)
Steps	1. System detects trigger event 2. Generates notification record 3. Notification displayed on student dashboard
Postcondition	Notification delivered and marked as unread

3. Design Overview

The HMS follows a three-tier client-server architecture: a React-based frontend, a Node.js + Express REST API backend, and a MySQL relational database. The system design prioritizes separation of concerns, maintainability, and a clean RESTful interface between the frontend and backend.

3.1 Design Goals and Constraints

Goals:

- Provide a role-based interface for Admins and Students
- Ensure all hostel data is stored persistently in a relational database
- Expose a stateless REST API for all backend operations
- Support future scalability for additional modules

Constraints:

- Technology Stack: React, Node.js, Express, MySQL (fixed by project requirements)
- Out of Scope: Fee payment and room booking by students
- Authentication: JWT-based session management
- Academic environment: No cloud deployment required; runs on localhost

3.2 Design Assumptions

- Each student is assigned to at most one room at a time
- Admin is the primary actor for all allocation and configuration tasks
- The system is used within a single hostel institution

- All users have stable internet connectivity in the deployment environment
- MySQL server is running and accessible on the same local network as the Node.js backend

3.3 Significant Design Packages

Package / Layer	Technology	Responsibility
Frontend (View Layer)	React.js	Renders UI, handles user interactions, calls REST API
API Layer (Controller)	Node.js + Express	Receives HTTP requests, validates input, calls service logic
Service Layer (Business Logic)	Node.js modules	Implements core HMS rules (allocation, complaint flow, etc.)
Data Access Layer	MySQL + mysql2 npm package	CRUD operations on the HMS database tables
Database	MySQL	Persists all application data across 5 core tables

3.4 Dependent External Interfaces

The HMS does not depend on external third-party services. All interfaces are internal to the system.

External Application & Interface Name	Module Using It	Functionality / Description
MySQL Database Server	Data Access Layer (Node.js)	Provides persistent relational storage; accessed via mysql2 npm package using SQL queries
Browser (Chrome / Firefox)	React Frontend	Hosts and renders the React SPA; provides DOM and fetch API for HTTP calls to backend

3.5 Implemented Application External Interfaces

The HMS backend exposes a REST API consumed by the React frontend.

Interface Name	Module Implementing It	Functionality / Description
POST /api/auth/login	Auth Controller	Accepts credentials, returns JWT token
GET /api/students	Student Controller	Returns list of all registered students
POST /api/students	Student Controller	Registers a new student
GET /api/rooms	Room Controller	Returns available and occupied room list
POST /api/rooms/allocate	Room Allocation Controller	Allocates a room to a student
POST /api/complaints	Complaint Controller	Submits a new complaint

Interface Name	Module Implementing It	Functionality / Description
GET /api/complaints	Complaint Controller	Returns all complaints (admin) or own complaints (student)
GET/POST /api/tickets	Ticket Controller	Manage maintenance tickets
GET/POST /api/notices	Notice Controller	Read and post notice board items
GET /api/notifications	Notification Controller	Returns notifications for the logged-in user

4. Logical View

This section describes the detailed software design, decomposed into modules and the key classes/components within each module.

4.1 Design Model

4.1.1 Frontend Components (React)

Component	Responsibility
App.jsx	Root component; manages routing and authentication state
LoginPage.jsx	Handles login form and JWT storage
Dashboard.jsx	Admin/Student landing page with navigation
StudentRegistration.jsx	Form for registering new students
RoomAllocation.jsx	View rooms, allocate/deallocate students
ComplaintForm.jsx	Student complaint submission form
ComplaintList.jsx	Admin view of all complaints with status update
TicketManagement.jsx	Ticket listing, assignment, and status update
NoticeBoard.jsx	Displays notices; admin can add new notices
NotificationPanel.jsx	Shows unread notifications for logged-in user
ProtectedRoute.jsx	HOC that restricts routes to authenticated users

4.1.2 Backend Modules (Node.js + Express)

Module	Responsibility
server.js	Entry point; initializes Express app, middleware, and routes
routes/auth.js	Defines /api/auth endpoints

Module	Responsibility
routes/students.js	Defines /api/students endpoints
routes/rooms.js	Defines /api/rooms endpoints
routes/complaints.js	Defines /api/complaints endpoints
routes/tickets.js	Defines /api/tickets endpoints
routes/notices.js	Defines /api/notices endpoints
routes/notifications.js	Defines /api/notifications endpoints
controllers/*.js	Business logic for each domain module
middleware/auth.js	JWT verification middleware applied to protected routes
config/db.js	MySQL connection pool setup using mysql2

4.2 Use Case Realization

4.2.1 UC-01: Student Registration – Sequence

1. Admin fills StudentRegistration form and clicks Submit.
2. React sends POST /api/students with student data in request body.
3. auth.js middleware verifies JWT token.
4. StudentController.createStudent() validates fields and calls DB insert.
5. MySQL inserts new record into Students table.
6. Controller returns 201 Created with student ID.
7. React displays success toast notification.

4.2.2 UC-02: Room Allocation – Sequence

1. Admin selects student and available room in RoomAllocation component.
2. React sends POST /api/rooms/allocate with { studentId, roomId }.
3. RoomController.allocateRoom() checks room availability.
4. If available: inserts into Allocations table; updates Rooms.status to 'OCCUPIED'.
5. Returns 200 OK; React refreshes room list.

4.2.3 UC-03: Complaint Submission – Sequence

1. Student fills ComplaintForm and submits.
2. React sends POST /api/complaints with { title, description, category }.
3. ComplaintController.createComplaint() sets status to PENDING, records studentId from JWT.
4. MySQL inserts into Complaints table.
5. NotificationController creates a notification record for admin.

6. Returns 201 Created.

5. Data View

5.1 Domain Model

The HMS domain consists of five core entities: Students, Rooms, Allocations, Complaints, and Tickets. A Student may have one active Allocation to a Room. Complaints are raised by Students and may spawn Tickets for resolution. The Notice Board and Notifications are additional entities associated with the notification flow.

5.2 Data Model (Persistent Data View)

5.2.1 Data Dictionary

Table: Students

Column	Type	Constraints	Description
student_id	INT	PK, AUTO_INCREMENT	Unique student identifier
name	VARCHAR(100)	NOT NULL	Full name of the student
roll_number	VARCHAR(50)	UNIQUE, NOT NULL	Academic roll number
email	VARCHAR(100)	UNIQUE, NOT NULL	Student email address
phone	VARCHAR(15)	NOT NULL	Contact number
password_hash	VARCHAR(255)	NOT NULL	Hashed password for login
created_at	DATETIME	DEFAULT NOW()	Record creation timestamp

Table: Rooms

Column	Type	Constraints	Description
room_id	INT	PK, AUTO_INCREMENT	Unique room identifier
room_number	VARCHAR(20)	UNIQUE, NOT NULL	Room label (e.g. A-101)
capacity	INT	NOT NULL	Maximum occupants
status	ENUM	DEFAULT 'AVAILABLE'	AVAILABLE or OCCUPIED
floor	INT	NOT NULL	Floor number

Table: Allocations

Column	Type	Constraints	Description
allocation_id	INT	PK, AUTO_INCREMENT	Unique allocation record ID
student_id	INT	FK -> Students.student_id	Allocated student
room_id	INT	FK -> Rooms.room_id	Allocated room
allocation_date	DATE	NOT NULL	Date of allocation
vacate_date	DATE	NULLABLE	Date student vacated (null if active)

Table: Complaints

Column	Type	Constraints	Description
complaint_id	INT	PK, AUTO_INCREMENT	Unique complaint ID
student_id	INT	FK -> Students.student_id	Student who raised complaint
title	VARCHAR(150)	NOT NULL	Complaint title
description	TEXT	NOT NULL	Detailed complaint description
category	VARCHAR(50)	NOT NULL	e.g. Electrical, Plumbing, Hygiene
status	ENUM	DEFAULT 'PENDING'	PENDING, IN_PROGRESS, RESOLVED
created_at	DATETIME	DEFAULT NOW()	Submission timestamp

Table: Tickets

Column	Type	Constraints	Description
ticket_id	INT	PK, AUTO_INCREMENT	Unique ticket ID
complaint_id	INT	FK -> Complaints.complaint_id, NULLABLE	Associated complaint (if any)
assigned_to	VARCHAR(100)	NULLABLE	Staff member assigned
priority	ENUM	DEFAULT 'MEDIUM'	LOW, MEDIUM, HIGH
status	ENUM	DEFAULT 'OPEN'	OPEN, IN_PROGRESS, CLOSED
created_at	DATETIME	DEFAULT NOW()	Ticket creation timestamp
resolved_at	DATETIME	NULLABLE	Resolution timestamp

6. Exception Handling

The HMS employs a centralized exception handling approach on the backend.

Exception / Error	Trigger Condition	Handling Strategy	HTTP Response
ValidationError	Missing or invalid request fields	Controller validates input; returns descriptive error message	400 Bad Request
AuthenticationError	Missing or invalid JWT token	auth middleware rejects request	401 Unauthorized
AuthorizationError	User lacks role permission	Controller checks role; rejects if unauthorized	403 Forbidden
NotFoundError	Resource (student, room, complaint) not found	Controller returns not-found message	404 Not Found
ConflictError	Duplicate roll number, room already occupied	Detected before DB insert; descriptive error returned	409 Conflict
DatabaseError	MySQL query failure or connection issue	Caught in try-catch; generic error returned; full error logged server-side	500 Internal Server Error
UnhandledError	Unexpected runtime exception	Global Express error handler catches and logs; returns generic message	500 Internal Server Error

All exceptions are logged to the server console with timestamps. In production, a logging library (e.g., Winston) should be integrated.

7. Configurable Parameters

The following parameters are configured via a .env file in the Node.js backend root directory.

Configuration Parameter	Definition and Usage	Dynamic?
PORT	Port on which the Express server listens (default: 5000)	No
DB_HOST	MySQL server hostname (default: localhost)	No
DB_USER	MySQL username for the HMS database connection	No
DB_PASSWORD	MySQL password for the HMS database connection	No
DB_NAME	Name of the MySQL database (default: hostel_db)	No
JWT_SECRET	Secret key used to sign and verify JWT tokens	No
JWT_EXPIRY	Token expiration duration (e.g., '1d', '8h')	No

Configuration Parameter	Definition and Usage	Dynamic?
NODE_ENV	Environment flag: development or production	No

8. Quality of Service

8.1 Availability

The HMS is designed for use within an academic institution's local network. The system targets 99% uptime during hostel operating hours (6:00 AM – 11:00 PM). As this is an academic project running on localhost, no high-availability infrastructure (e.g., load balancers) is required. The MySQL server and Node.js backend should be started before any users access the system. Database backups should be performed at least once daily.

8.2 Security and Authorization

- Authentication: All users (Admin and Student) authenticate via username/password; a JWT token is issued upon successful login and must accompany all API requests via the Authorization header.
- Role-Based Access: Admin routes (room allocation, complaint management, notice posting) are protected by role checks in the middleware. Students can only access their own data.
- Password Storage: Passwords are hashed using bcrypt before storage; plaintext passwords are never stored.
- Input Sanitization: All user inputs are validated on the backend to prevent SQL injection and XSS.

8.3 Load and Performance Implications

Given the academic scope (single hostel, estimated 50–200 concurrent users maximum), the system is not expected to face significant load challenges. Key design decisions to support acceptable performance:

- MySQL connection pooling (via mysql2) is used to reuse database connections and reduce overhead.
- React frontend uses component-level state management (useState/useEffect) to avoid unnecessary re-renders.
- API responses are kept minimal (no over-fetching); only required fields are returned.
- For future scaling: indexes should be placed on student_id, room_id, and complaint_id foreign keys.

8.4 Monitoring and Control

In the academic deployment context, monitoring is handled through:

- Console Logging: All API requests and errors are logged to the Node.js server console.
- Error Tracking: Unhandled exceptions are caught by the global Express error handler and logged with timestamps.
- Database Health: MySQL server status can be verified manually via the MySQL CLI or a tool like MySQL Workbench.
- Future Enhancement: Integration with a logging framework (Winston, Morgan) and a monitoring dashboard would be recommended for production deployment.